

## CONTENTS

- Overview
- 1 The Hardware Store Analogy: Two Kinds of AI Systems
- 2 ADK: Designing AI That Acts and Reasons
- 3 RAG: Designing AI That Retrieves and Knows
- 4 Choosing ADK: When Your AI Needs To Do Things
- 5 Choosing RAG: When Your Data Is the Source of Truth
- 6 Hybrid Systems: Reasoning Paired With Domain Knowledge
- 7 A Practical Decision Checklist
- Glossary
- Check yourself
- Where to go next

# ADK vs RAG: How to Choose the Right AI Stack

Use a simple test — should the AI act or should it recall — to decide between an agent architecture, a retrieval pipeline, or both.

For developers and technical product people deciding whether to build an AI agent, a retrieval system, or a combination of the two · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

## BEFORE YOU START

- Basic understanding of what a large language model (LLM) does when it generates text
- Familiarity with the idea of calling a function or API from code

---

## Overview

Every AI system design eventually runs into the same fork: should the system perform actions, or should it answer questions grounded in a body of knowledge? Agent architectures (often built with an Agent Development Kit, or ADK) are optimized for the first job — following instructions, calling tools, and executing multi-step workflows. Retrieval-Augmented Generation (RAG) is optimized for the second — grounding answers in documents so the model isn't relying on guesswork baked into its training data.

These aren't competing technologies so much as answers to different questions. An agent framework gives a language model hands: it can call a function, follow a decision tree, and keep doing that reliably every time it's asked. A retrieval pipeline gives a language model a library card: before it answers, it looks something up, so its response reflects your documents instead of whatever it happened to memorize during training.

Most production systems that look sophisticated from the outside are not choosing one over the other — they're combining them. An agent orchestrates the steps of a task, and one or more of those steps calls a retrieval pipeline to fetch grounded facts before continuing. Understanding when each piece is doing the work, and why, is what makes that combination deliberate instead of accidental.

#### KEY TAKEAWAYS

- ✓ Agent architectures (ADK-style) are for AI that needs to act: follow steps, call tools, and produce consistent, repeatable behavior.
- ✓ RAG is for AI that needs to know: it retrieves relevant text from your documents before generating an answer, so the answer is grounded in that source instead of the model's internal training.
- ✓ The fastest way to choose is to ask one question: does this system need to act, or does it need to recall?
- ✓ ADK suits multi-step workflows, tool use, and tasks where the value comes from reasoning and repeatable logic, not memory.
- ✓ RAG suits domains where the source of truth is a document set that is large, detailed, or changes often — retrieval keeps answers current without retraining a model.
- ✓ Hybrid systems are the norm in real deployments: the agent layer handles task flow and decisions, while a retrieval layer supplies it with accurate, up-to-date facts.
- ✓ A retrieval step and a tool-calling step are not mutually exclusive — retrieval can itself be exposed to an agent as just another tool it decides to call.

## 01 The Hardware Store Analogy: Two Kinds of AI Systems

0:00

A hardware store has a tool aisle for doing work and a reference aisle for knowing how to do it correctly — AI systems split the same way.

Walk into a hardware store to fix something and you'll notice two distinct aisles. One holds tools: drills, saws, sanders. You point a tool at a task and it performs the work directly. The other holds reference material: manuals, diagrams, buying guides. A manual does not drive a single screw for you, but it tells you which screw to use and how deep to sink it.

AI system design splits along the same line. Some AI is built to perform work directly — take an instruction, carry out a sequence of steps, and produce a result. Other AI is built to supply accurate knowledge — answer a question correctly by consulting a source, rather than acting on the world. Neither kind is more advanced than the other; they solve different problems, the same way a drill and a manual solve different problems in the same repair job.

The rest of this lesson names these two approaches: ADK-style agent architectures for the 'tool' role, and RAG for the 'reference' role. Keeping the analogy in mind makes the architectural decision concrete instead of abstract — you're not choosing a trendier technology, you're choosing the aisle that matches what the job actually requires.

#### KEY POINTS

- Tools act on the world; reference material informs decisions but doesn't act on its own.
- Neither approach is inherently more advanced — they answer different questions.
- The analogy maps directly onto two AI architecture styles covered next: ADK (acts) and RAG (knows).

### Fixing a bookshelf

Building a bookshelf needs a drill (a tool, acting) and possibly a joinery diagram (a reference, informing). You wouldn't try to drive a screw with the diagram, and you wouldn't expect the drill to tell you which screw to use. Most real carpentry projects, like most real AI systems, need both.

## 02 ADK: Designing AI That Acts and Reasons

0:43

ADK-style systems give an AI model the ability to reason step by step and call tools to complete multi-step tasks predictably.

ADK stands for Agent Development Kit, and the term is used here for a broader pattern: agent architectures built around action and reasoning. Instead of producing a single block of text as its final output, an ADK-style agent is put inside a loop. On each turn it can call a workflow, invoke a tool (a function, an API, a database query), follow an explicit rule or process, and then decide what to do next based on what came back.

The defining characteristics are step-by-step reasoning, use of external tools, adherence to rules or logic, and — importantly — consistent, repeatable behavior. Because the agent follows the same decision logic every time it's given a similar input, its behavior is easier to test and evaluate than a single freeform generation. This loop is sometimes called the ReAct pattern (reason, then act, then observe the result, then reason again), and it's the mechanical core of almost every agent framework, whether or not it's branded 'ADK'.

ADK works well specifically when your AI needs to perform a task in a predictable sequence — draft a document, triage a support ticket, fill out a form, coordinate a handoff between systems. The value it provides comes from reasoning through steps and taking action, not from remembering facts. If you asked an ADK-style agent 'what does section 4.2 of our contract say,' it has no built-in way to know — that's not the job it's built for.

## KEY POINTS

- ADK-style agents run in a loop: reason about the next step, call a tool, observe the result, reason again.
- Core traits: step-by-step reasoning, tool use, explicit rules/logic, and repeatable behavior across runs.
- Repeatability is what makes agent behavior testable — the same input tends to trigger the same decision path.
- The agent's value is in reasoning and acting, not in recalling facts it was never given.

## A minimal tool-calling agent loop

This is the mechanical skeleton every ADK-style agent shares: give the model a set of callable tools, let it decide whether to call one or answer directly, feed the tool's result back in, and repeat until it has enough information to respond. The 'llm' and 'tools' objects stand in for a real model client and real functions.

```
def run_agent(user_request, tools, llm, max_steps=6):
    """A minimal ReAct-style agent loop: reason, act, observe, repeat."""
    messages = [{"role": "user", "content": user_request}]

    for step in range(max_steps):
        response = llm.chat(messages, tools=tools)

        if response.tool_call is None:
            # Model decided it has enough information to answer directly
            return response.content

        # Execute the tool the model asked for
        tool_name = response.tool_call.name
        tool_args = response.tool_call.arguments
        result = tools[tool_name](**tool_args)

        # Feed the observation back so the model can reason further
        messages.append({"role": "assistant", "tool_call": response.tool_call})
        messages.append({"role": "tool", "name": tool_name, "content": str(result)})

    return "Max steps reached without a final answer."
```

RAG connects a model to your documents and retrieves the relevant pieces before it answers, trading the model's internal guesswork for grounded, sourced text.

RAG stands for Retrieval-Augmented Generation. Where ADK focuses on action, RAG focuses on knowledge and accuracy. Instead of relying on whatever a model absorbed during training, a RAG system connects to your actual documents, retrieves the specific passages relevant to the current question, and only then generates a response — using those passages as its source material.

Mechanically, this means every document is broken into chunks and converted into embeddings: numeric vectors that place similar meanings near each other in a high-dimensional space. Those vectors sit in a vector database. When a question arrives, it's embedded the same way, and the database returns the chunks whose vectors are closest to the question's vector. Those chunks get inserted into the prompt as context, and the model is instructed to answer using that context rather than its own memory.

The payoff is groundedness: the answer traces back to a specific document, and updating that document changes future answers immediately, with no retraining. This matters most exactly where a model's memorized knowledge is unreliable — information that's high-volume, highly detailed, or that changes on a schedule a training run can't keep up with.

#### KEY POINTS

- RAG retrieves relevant document chunks before generating a response, instead of answering from memorized training data.
- Documents are chunked and embedded into vectors; a vector database finds chunks whose meaning is closest to the question.
- Retrieved chunks are inserted into the prompt as context that the model is told to rely on.
- Updating the underlying documents changes future answers immediately — no retraining required.

## A minimal RAG answer function

This function embeds the incoming question, retrieves the top-k most relevant chunks from a vector store, builds a context block from them, and instructs the model to answer only from that context. If nothing relevant was retrieved, the model is told to say so rather than fill the gap with a guess.

```
def answer_with_rag(question, vector_store, llm, top_k=4):
    """Retrieve relevant document chunks, then generate a grounded answer."""
    query_embedding = embed(question)
    chunks = vector_store.search(query_embedding, top_k=top_k)

    context = "\n\n".join(f"[{c.source}] {c.text}" for c in chunks)
    prompt = (
        "Answer using ONLY the context below. "
        "If the answer isn't there, say so.\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )
    return llm.complete(prompt)
```

04

## Choosing ADK: When Your AI Needs To Do Things

2:51

The core test is 'act or recall?' — ADK says 'do something for me,' and it fits anywhere the value comes from reasoning through steps rather than looking something up.

With both approaches defined, the choice comes down to one question: is this AI meant to act, or is it meant to recall? An ADK-style agent says 'do something for me.' A RAG system says 'tell me something about my data.' Nearly every architectural decision downstream of this point follows from which of those two sentences describes the task.

Choose ADK when you want the system to be procedural — when there's a sequence of steps that needs to happen in order, regardless of what any single document says. Typical use cases include multi-step workflows, drafting or transforming content, IT or HR assistance, and task coordination or triage, where an incoming request has to be routed, classified, or escalated according to a rule set.

The other reason to choose ADK is reliability of behavior. Because the agent follows the same logic on every run, its outputs are far more consistent than a single open-ended generation, which makes evaluation and debugging tractable — you can point at the exact step where a run diverged from what you expected, because the steps are explicit and repeatable.

#### KEY POINTS

- The one-question test: does the AI need to act (ADK) or recall (RAG)?
- ADK fits multi-step workflows, content drafting/transformation, IT/HR assistance, and triage or routing tasks.
- Choose ADK when the value comes from reasoning through a decision, not from looking up a fact.
- Consistent, repeatable logic makes ADK-style systems easier to evaluate than free-form generation.

### IT support triage as a procedural task

A request like 'my VPN keeps disconnecting' needs to be classified (network vs. account issue), routed to the right queue, and possibly have a ticket created — the same three steps every time, regardless of which specific document might describe VPN troubleshooting. That repeatable sequence is exactly what an ADK-style agent is built to execute.

## 05 Choosing RAG: When Your Data Is the Source of Truth

3:37

Choose RAG when accuracy has to come directly from documents, especially where content is high-volume, detailed, or constantly changing.

RAG is the right choice when your data is the source of truth and the model's own internal knowledge isn't good enough to rely on — either because it was never trained on that material, or because the material has since changed. Ideal content for RAG includes PDFs, policies and regulations, technical documentation, product manuals, and long-form knowledge bases: exactly the kind of material a human would flip through rather than memorize.

RAG shines specifically where a system needs to 'remember' things no person could reasonably hold in their head — high volume, high detail, and frequently updated information. It also handles open-ended questions well, the kind where the exact phrasing varies each time: 'where is this topic mentioned,' 'what does this report say,' 'summarize this section.' A fixed workflow can't anticipate every phrasing in advance, but retrieval doesn't need to — it just needs the question's meaning to be close, in embedding space, to the relevant passage.

Common use cases follow directly from this: knowledge search across internal wikis, research assistance, legal or medical document lookup, and technical support that has to be grounded in actual documentation rather than a support agent's best guess.



#### KEY POINTS

- Use RAG when accuracy must trace back to a document, not to the model's internal guesswork.
- Best-fit content: PDFs, policies, regulations, technical docs, product manuals, long-form knowledge bases.
- RAG handles information too large or detailed for a person (or a fixed prompt) to hold in full.
- It also handles widely varying question phrasing, since retrieval matches on meaning rather than a fixed script.

### Legal document lookup

A question like 'does our vendor contract with Acme allow termination for convenience' can't be answered from a model's training — it depends on the specific contract text. A RAG system retrieves the relevant clause from the actual contract PDF and answers from that clause, rather than producing a plausible-sounding but unverified guess.

## 06 Hybrid Systems: Reasoning Paired With Domain Knowledge

4:20

Most real systems use both: an ADK-style agent handles the task flow and decisions, while a RAG component supplies it with grounded facts along the way.

Most real-world AI systems don't strictly pick one architecture — they combine them. In a hybrid system, the ADK layer handles task flow, decision logic, and the sequence of steps, while the RAG layer supplies accurate information pulled from documents whenever a step in that sequence needs a fact. The result is a system that is both intelligent (it can reason and act) and well-informed (its answers are grounded in real sources).

A useful way to place a system on this map: content generation with low retrieval need and high reasoning need points to ADK. Internal knowledge search where the answer depends entirely on documents points to RAG. Automation, IT assistance, and multi-step tool use point to ADK. Domain-expert co-pilot work that needs both deep retrieval and complex reasoning — legal and engineering co-pilots, healthcare assistants, enterprise task co-pilots — is where the hybrid approach earns its complexity.

The cleanest way to build a hybrid system is not to bolt RAG on beside the agent, but to expose retrieval to the agent as just another tool. The agent's reasoning loop decides when a step requires grounded information, calls the retrieval function like any other tool, and folds the result back into its next decision. This keeps a single control loop in charge of the whole task, with RAG as one capability inside it rather than a separate, disconnected system.

### KEY POINTS

- Hybrid = ADK controls the task flow and decisions; RAG supplies grounded facts at the steps that need them.
- Content generation / high reasoning / low retrieval -> ADK. Answers dependent entirely on documents -> RAG.
- Domain-expert co-pilots (legal, engineering, healthcare) typically need both deep retrieval and complex reasoning.
- The cleanest integration exposes retrieval to the agent as one more callable tool, not a separate parallel system.

## A support agent with retrieval as a tool

The agent loop from the ADK section is reused unchanged. One of its tools, 'retrieve\_docs', is simply the RAG function from the previous section. The agent decides on its own when a step needs a grounded fact and calls that tool, then continues reasoning and acting with whatever it learned.

```
tools = {
    "retrieve_docs": lambda query: answer_with_rag(query, vector_store, llm),
    "create_ticket": create_ticket,
    "send_email": send_email,
}

run_agent(
    "A customer is asking about our refund policy for damaged items. "
    "Look it up and draft a reply.",
    tools=tools,
    llm=llm,
)
```

## 07 A Practical Decision Checklist

5:42

Back to the hardware store: use the tool aisle to build, the reference aisle to make sure the build is correct, and pick based on whether the task is to act, to know, or both.

Back to the hardware store one more time: ADK is the tool aisle — it performs the steps, it acts, it builds. RAG is the reference aisle — it provides information and grounds the work in facts. Most successful projects use both aisles: tools to build, guides to make sure the build is correct.

In practice, the decision reduces to two yes/no questions. Does the system need to take actions or follow a multi-step process (needs\_action)? Does the system need to answer from a specific, possibly changing, body of documents (needs\_current\_docs)? The answers place the system in one of four positions: pure ADK, pure RAG, a hybrid of both, or — for the simplest cases — a direct model call with no architecture on top at all.

Answering those two questions first, before picking a framework or a vendor, is what keeps the architecture matched to the job. The mistake this lesson is built to prevent is building an elaborate retrieval pipeline for a task that was purely procedural, or building a rigid multi-step agent for a task that was really just 'go look this up.'

#### KEY POINTS

- ADK = tool aisle (acts, builds); RAG = reference aisle (informs, grounds).
- Two questions decide the architecture: does it need to act? Does it need current, document-grounded knowledge?
- Four possible outcomes: ADK only, RAG only, hybrid, or a plain model call with no extra architecture.
- Deciding this before picking a framework prevents over- or under-building the system.

## A two-question decision function

A direct translation of the hardware-store heuristic into code. It's deliberately simple — the point isn't the code, it's that the two questions are enough to place almost any system into the right quadrant before any framework gets chosen.

```
def choose_architecture(needs_action, needs_current_docs):
    if needs_action and needs_current_docs:
        return "Hybrid: ADK orchestrates the task, RAG supplies grounded facts"
    if needs_action:
        return "ADK: a procedural, tool-using agent"
    if needs_current_docs:
        return "RAG: retrieval over your documents"
    return "A plain model call may be enough"

# Examples from the lesson
print(choose_architecture(needs_action=True, needs_current_docs=False)) # IT triage
print(choose_architecture(needs_action=False, needs_current_docs=True))  # legal lookup
print(choose_architecture(needs_action=True, needs_current_docs=True))   # legal co-
pilot
```

## Glossary

### **ADK (Agent Development Kit)**

A term for agent-style architectures built around action and reasoning — the AI follows steps, calls tools, and produces consistent, repeatable behavior.

### **RAG (Retrieval-Augmented Generation)**

An architecture where a model retrieves relevant text from a document store before generating its answer, so the answer is grounded in that source material.

### **Agent**

An AI system that runs in a loop, deciding on each turn whether to call a tool or produce a final answer, rather than generating one response in isolation.

### **Tool use / function calling**

The mechanism by which a model can invoke an external function or API mid-conversation and receive its result back as new information to reason over.

### **Embedding**

A numeric vector representation of a piece of text, positioned so that text with similar meaning ends up close together in vector space.

### **Vector database**

A database optimized for storing embeddings and finding the ones closest in meaning to a given query vector.

### **Grounding**

Basing a model's answer on retrieved source material rather than on patterns memorized during training, so the answer can be traced back to a specific document.

### **ReAct pattern**

An agent loop of reason, then act, then observe the result, then reason again — the mechanical basis of most tool-calling agents.

## Triage

Classifying and routing an incoming request (a ticket, a message) to the right handler or process based on rules or logic.

## Hybrid system

An architecture where an agent (ADK-style) controls the task flow and calls a retrieval pipeline (RAG) as one of its tools whenever a step needs grounded information.

## Check yourself

Answer first, then open each one.

**A support bot answers questions about a 200-page policy manual that gets revised every month. Why would a RAG system stay accurate after a revision while a plain fine-tuned model would not?**

RAG retrieves from the live document at answer time, so an edit to the source document is reflected in the very next query. A fine-tuned model bakes the old text into its weights at training time; it keeps answering from the stale version until it's retrained, which is slower and more expensive than just updating a document.

**Why is an ADK-style agent easier to evaluate than a single free-form model generation?**

Because the agent follows explicit, repeatable steps (reason, call a tool, observe, repeat), the same input tends to produce the same decision path each run. That repeatability lets you inspect and test individual steps, whereas a single open-ended generation gives you no intermediate points to check.

**In the hybrid support-agent example, retrieval is exposed to the agent as just another tool rather than as a separate pipeline running alongside it. Why does that design choice matter?**

Keeping retrieval inside the agent's own tool set means one control loop decides when grounded information is actually needed for the current step, instead of always running retrieval whether or not the step calls for it. It also means the retrieved result flows back into the agent's next reasoning step naturally, the same way any other tool result would.

**A multi-step onboarding workflow (create account, assign role, send welcome email) doesn't reference any documents. Why would adding a RAG layer here likely be wasted effort?**

The task's value comes entirely from executing a fixed sequence correctly, not from looking anything up — there's no changing, document-based fact the workflow needs to consult. RAG solves a knowledge problem, and this task doesn't have one.

**Why does 'does the answer change when I update a document' work as a quick test for whether a system needs RAG?**

If updating a document should change future answers, the system's correctness depends on live document content, not on task logic — which is exactly the situation retrieval is built for. If the answer wouldn't change no matter what the document said, the system is really about executing steps, which points to an agent architecture instead.

## Where to go next

- Build a minimal ReAct-style tool-calling loop in Python against a real LLM API and give it two or three simple tools (a calculator, a date lookup, a mock ticket creator).
- Implement a small RAG pipeline end to end: chunk a handful of PDFs, embed them with an open-source embedding model, store the vectors in a local vector database (e.g. Chroma or FAISS), and query it.
- Take one real workflow you're considering automating and run it through the two-question checklist (needs\_action, needs\_current\_docs) before choosing a framework.
- Look up Google's Agent Development Kit documentation to see a concrete, production implementation of the agent loop pattern described in this lesson.
- Read about the ReAct paper (Yao et al., 2022) for the original formulation of the reason-act-observe loop underlying most modern tool-calling agents.

---

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.